Comparison: Credit Card vs PaySim

This notebook creates a **separate advanced comparison section** for the diploma:

- clean comparison tables
- ranking tables
- best model summary
- advanced charts
- exported CSV and Excel files

1. Imports

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2. Load result files

This notebook expects these files:

- `creditcard_final_results.csv`
- `paysim_final_results.csv`

Run the dataset notebooks first so these files are created.

```
def find_existing_file(candidates):
    for path in candidates:
        if os.path.exists(path):
            return path
    return None

credit_candidates = [
    "creditcard_final_results.csv",
    "/content/creditcard_final_results.csv",
    "/mnt/data/creditcard_final_results.csv"
]

paysim_candidates = [
    "paysim_final_results.csv",
    "/content/paysim_final_results.csv",
    "/mnt/data/paysim_final_results.csv"
]

credit_path = find_existing_file(credit_candidates)
paysim_path = find_existing_file(paysim_candidates)
```

```
print("Credit Card results file:", credit_path)
print("PaySim results file:", paysim_path)

if credit_path is None or paysim_path is None:
    raise FileNotFoundError(
        "Required CSV results files were not found. "
        "Please run the Credit Card and PaySim notebooks first."
    )

credit_df = pd.read_csv(credit_path)
paysim_df = pd.read_csv(paysim_path)

print("Credit Card shape:", credit_df.shape)
print("PaySim shape:", paysim_df.shape)
```

```
Credit Card results file: creditcard_final_results.csv
PaySim results file: paysim_final_results.csv
Credit Card shape: (4, 13)
PaySim shape: (4, 6)
```

3. Clean and standardize tables

```
metric_cols = ["Precision", "Recall", "F1-score", "ROC-AUC", "PR-AUC"]

for df in [credit_df, paysim_df]:
    for col in metric_cols:
        df[col] = pd.to_numeric(df[col], errors="coerce")

credit_df["Dataset"] = "Credit Card"
paysim_df["Dataset"] = "PaySim"

combined_df = pd.concat([credit_df, paysim_df], ignore_index=True)
combined_df = combined_df.sort_values(["Dataset", "F1-score"], ascending=[Tr

combined_df.head()
```

	Dataset	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC	
0	Credit Card	XGBoost	0.884211	0.857143	0.870466	0.968003	0.875352	5685
1	Credit Card	Isolation Forest	0.759887	0.546748	0.635934	0.955073	0.649666	5677
2	Credit Card	Autoencoder	0.653846	0.414634	0.507463	0.938165	0.555035	5675
3	Credit Card	Ensemble	0.990000	0.201220	0.334459	0.992524	0.966918	5686

Next steps: Generate code with `combined_df` New interactive sheet

4. Separate comparison tables for each dataset

```
credit_comparison = credit_df[["Model", "Precision", "Recall", "F1-score", "
credit_comparison = credit_comparison.sort_values("F1-score", ascending=Fals
credit_comparison.insert(0, "Rank", range(1, len(credit_comparison) + 1))

paysim_comparison = paysim_df[["Model", "Precision", "Recall", "F1-score", "
paysim_comparison = paysim_comparison.sort_values("F1-score", ascending=Fals
paysim_comparison.insert(0, "Rank", range(1, len(paysim_comparison) + 1))

print("Credit Card comparison table")
display(credit_comparison.round(4))

print("PaySim comparison table")
display(paysim_comparison.round(4))
```

```
Credit Card comparison table
```

	Rank	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
0	1	XGBoost	0.8842	0.8571	0.8705	0.9680	0.8754
1	2	Isolation Forest	0.7599	0.5467	0.6359	0.9551	0.6497
2	3	Autoencoder	0.6538	0.4146	0.5075	0.9382	0.5550
3	4	Ensemble	0.9900	0.2012	0.3345	0.9925	0.9669

```
PaySim comparison table
```

	Rank	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
0	1	XGBoost	0.3834	0.9625	0.5484	0.9992	0.9306
1	2	Autoencoder	0.6993	0.4510	0.5484	0.9383	0.5362
2	3	Isolation Forest	0.5452	0.2677	0.3591	0.8574	0.2829
3	4	Ensemble	1.0000	0.2008	0.3344	0.9991	0.9879

5. Combined advanced comparison table

```
advanced_comparison = combined_df[["Dataset", "Model", "Precision", "Recall"
advanced_comparison = advanced_comparison.sort_values(["Dataset", "F1-score"
advanced_comparison_round = advanced_comparison.round(4)

advanced_comparison_round
```

	Dataset	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
0	Credit Card	XGBoost	0.8842	0.8571	0.8705	0.9680	0.8754
1	Credit Card	Isolation Forest	0.7599	0.5467	0.6359	0.9551	0.6497
2	Credit Card	Autoencoder	0.6538	0.4146	0.5075	0.9382	0.5550
3	Credit Card	Ensemble	0.9900	0.2012	0.3345	0.9925	0.9669
4	PaySim	XGBoost	0.3834	0.9625	0.5484	0.9992	0.9306
5	PaySim	Autoencoder	0.6993	0.4510	0.5484	0.9383	0.5362

Next steps: Generate code with advanced_comparison_round | New interactive sheet

6. Best model summary

```
best_by_dataset = advanced_comparison.loc[
    advanced_comparison.groupby("Dataset")["F1-score"].idxmax()
].sort_values("Dataset").reset_index(drop=True)

overall_best = advanced_comparison.sort_values("F1-score", ascending=False).

print("Best model in each dataset:")
display(best_by_dataset.round(4))

print("Overall best model across both datasets:")
display(overall_best.round(4))
```

```
Best model in each dataset:
```

	Dataset	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
0	Credit Card	XGBoost	0.8842	0.8571	0.8705	0.9680	0.8754
1	PaySim	XGBoost	0.3834	0.9625	0.5484	0.9992	0.9306

```
Overall best model across both datasets:
```

	Dataset	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
0	Credit Card	XGBoost	0.8842	0.8571	0.8705	0.968	0.8754

7. Ranking comparison between datasets

```
rank_table = advanced_comparison.copy()
rank_table["Rank"] = rank_table.groupby("Dataset")["F1-score"].rank(method="
rank_table = rank_table[["Dataset", "Rank", "Model", "F1-score", "ROC-AUC",
rank_table = rank_table.sort_values(["Dataset", "Rank"]).reset_index(drop=Tr
```

```
rank_table.round(4)
```

	Dataset	Rank	Model	F1-score	ROC-AUC	PR-AUC
0	Credit Card	1	XGBoost	0.8705	0.9680	0.8754
1	Credit Card	2	Isolation Forest	0.6359	0.9551	0.6497
2	Credit Card	3	Autoencoder	0.5075	0.9382	0.5550
3	Credit Card	4	Ensemble	0.3345	0.9925	0.9669
4	PaySim	1	XGBoost	0.5484	0.9992	0.9306
5	PaySim	2	Autoencoder	0.5484	0.9383	0.5362
6	PaySim	3	Isolation Forest	0.3591	0.8574	0.2829
7	PaySim	4	Ensemble	0.3344	0.9991	0.9879

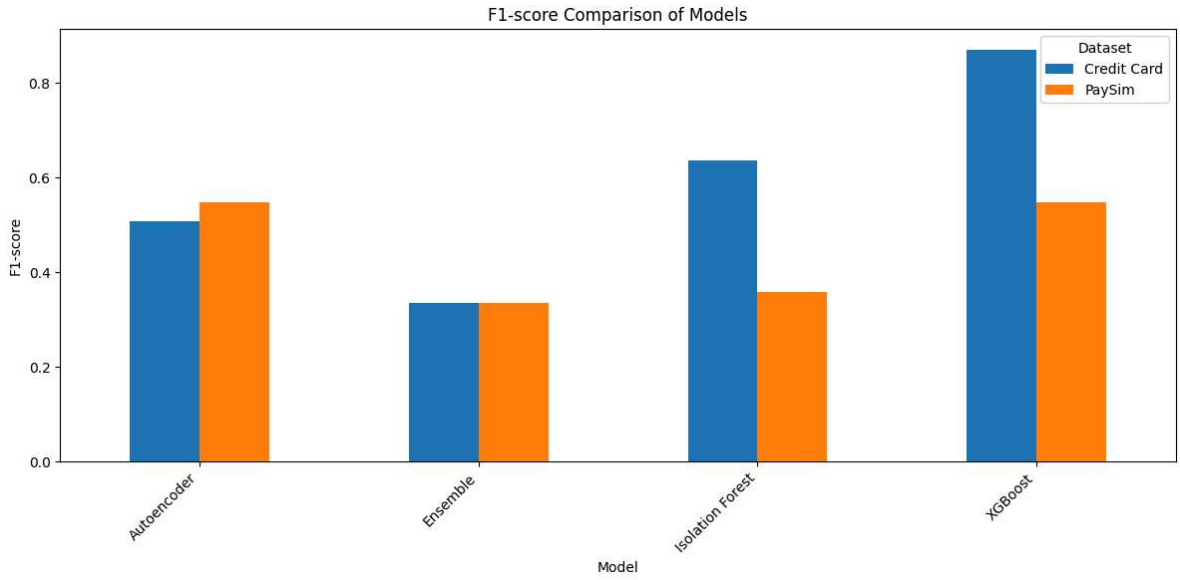
8. Export tables

```
credit_comparison.round(6).to_csv("creditcard_model_comparison_advanced.csv"
paysim_comparison.round(6).to_csv("paysim_model_comparison_advanced.csv", in
advanced_comparison_round.to_csv("combined_model_comparison_advanced.csv", i
rank_table.round(6).to_csv("model_ranking_comparison.csv", index=False)
best_by_dataset.round(6).to_csv("best_models_by_dataset.csv", index=False)

with pd.ExcelWriter("advanced_model_comparison.xlsx", engine="openpyxl") as
    credit_comparison.round(6).to_excel(writer, sheet_name="CreditCard", ind
    paysim_comparison.round(6).to_excel(writer, sheet_name="PaySim", index=F
    advanced_comparison_round.to_excel(writer, sheet_name="Combined", index=
    rank_table.round(6).to_excel(writer, sheet_name="Ranking", index=False)
    best_by_dataset.round(6).to_excel(writer, sheet_name="BestModels", index

print("Export complete.")
```

```
Export complete.
```

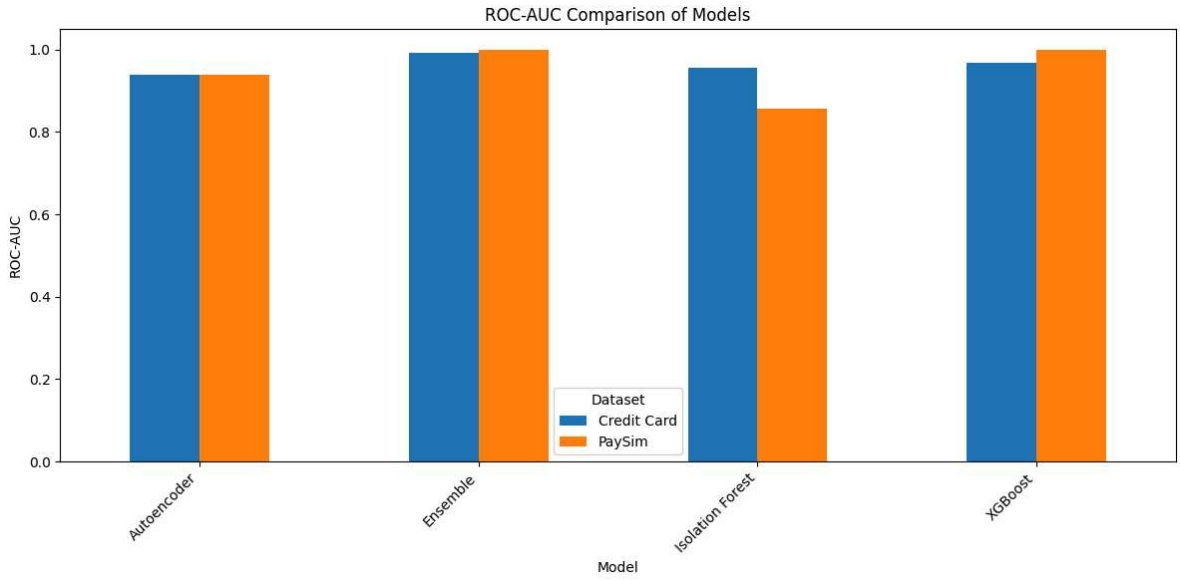
9. Chart 1 — F1-score comparison

```
pivot_f1 = advanced_comparison.pivot(index="Model", columns="Dataset", value
ax = pivot_f1.plot(kind="bar", figsize=(12, 6))
ax.set_title("F1-score Comparison of Models")
ax.set_xlabel("Model")
ax.set_ylabel("F1-score")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



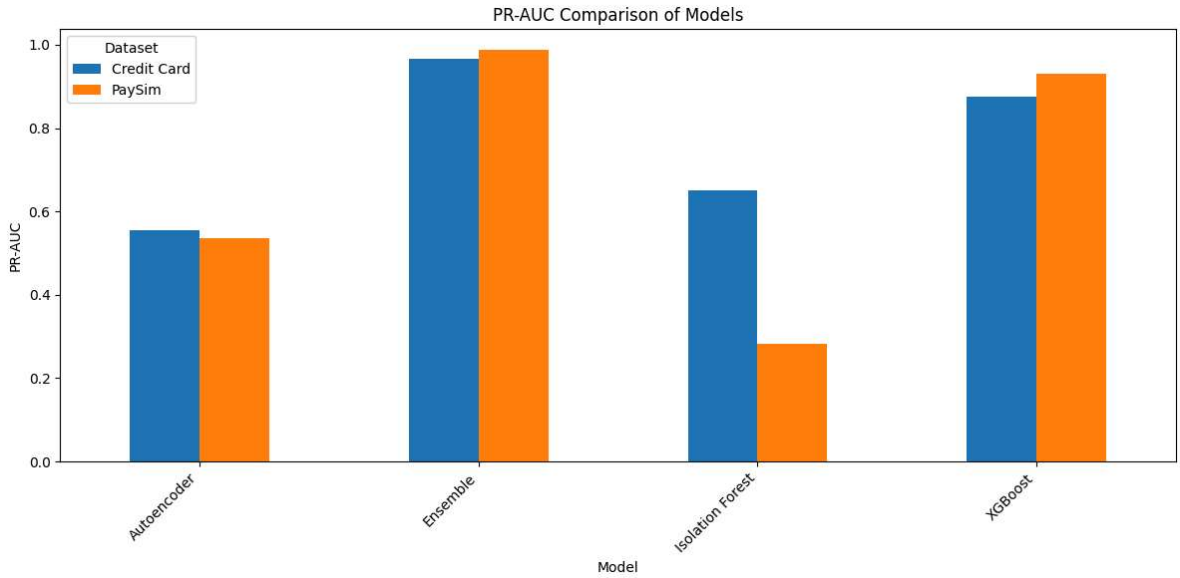
10. Chart 2 — ROC-AUC comparison

```
pivot_roc = advanced_comparison.pivot(index="Model", columns="Dataset", valu
ax = pivot_roc.plot(kind="bar", figsize=(12, 6))
ax.set_title("ROC-AUC Comparison of Models")
ax.set_xlabel("Model")
ax.set_ylabel("ROC-AUC")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



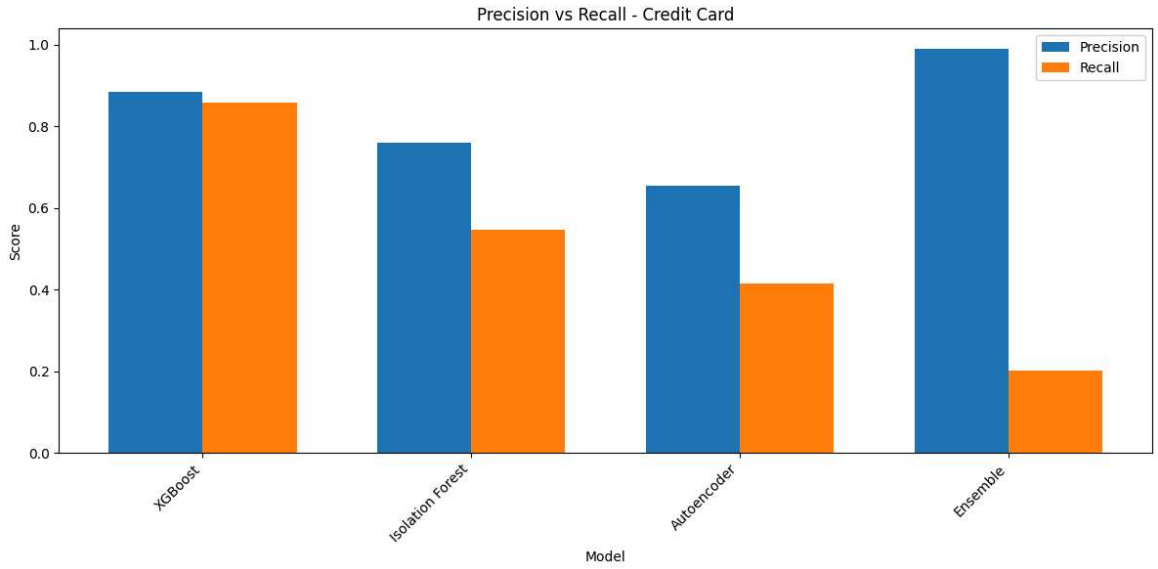
11. Chart 3 — PR-AUC comparison

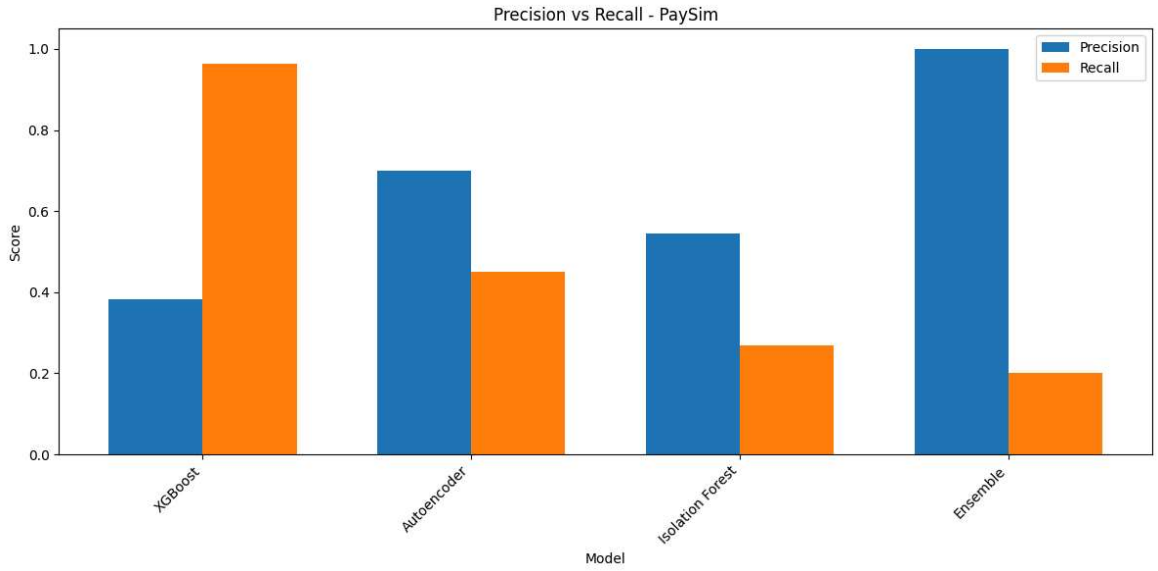
```
pivot_pr = advanced_comparison.pivot(index="Model", columns="Dataset", value
ax = pivot_pr.plot(kind="bar", figsize=(12, 6))
ax.set_title("PR-AUC Comparison of Models")
ax.set_xlabel("Model")
ax.set_ylabel("PR-AUC")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



12. Chart 4 — Precision and Recall by dataset

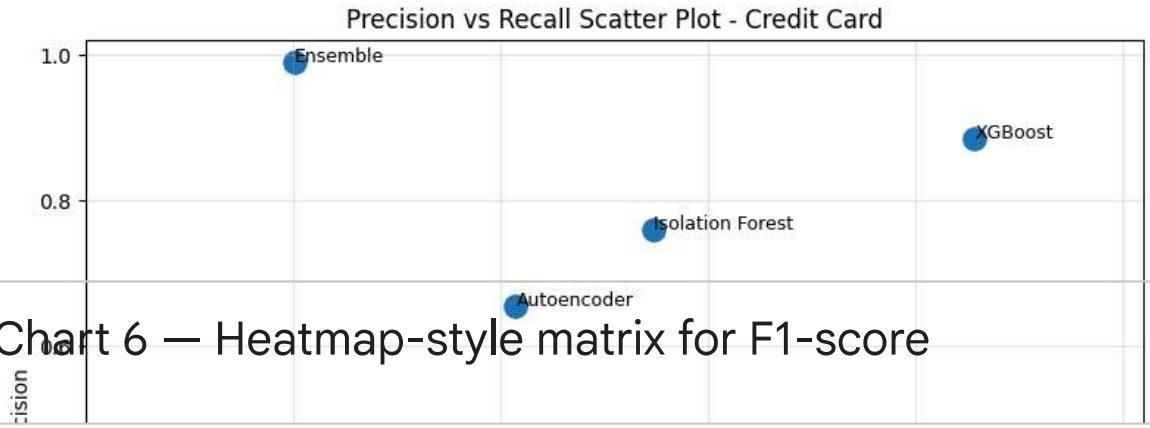
```
for dataset_name in advanced_comparison["Dataset"].unique():
    subset = advanced_comparison[advanced_comparison["Dataset"] == dataset_n
    subset = subset.sort_values("F1-score", ascending=False)

    x = np.arange(len(subset))
    width = 0.35

    plt.figure(figsize=(12, 6))
    plt.bar(x - width/2, subset["Precision"], width=width, label="Precision"
    plt.bar(x + width/2, subset["Recall"], width=width, label="Recall")
    plt.xticks(x, subset["Model"], rotation=45, ha="right")
    plt.title(f"Precision vs Recall - {dataset_name}")
    plt.xlabel("Model")
    plt.ylabel("Score")
    plt.legend()
    plt.tight_layout()
    plt.show()
```


Precision vs Recall - Credit Card
Precision
Recall
Score
1.0
0.8
0.6
0.4
0.2
0.0
XGBoost
Isolation Forest
Autoencoder
Ensemble
Model


Precision vs Recall - PaySim
Precision
Recall
Score
1.0
0.8
0.6
0.4
0.2
0.0
XGBoost
Autoencoder
Isolation Forest
Ensemble
Model

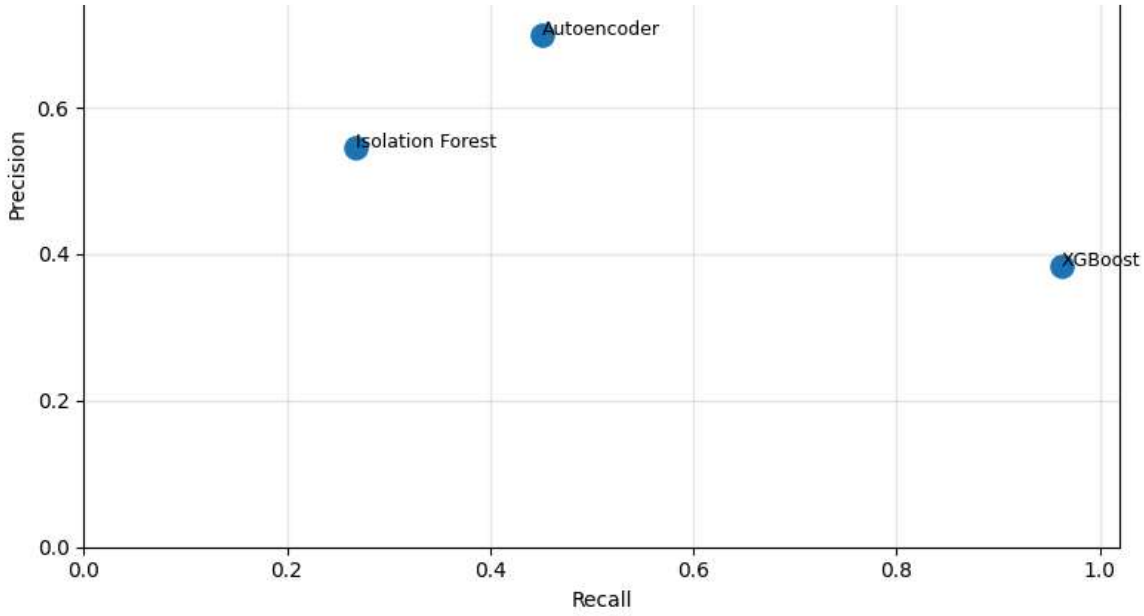
13. Chart 5 — Precision vs Recall scatter plot

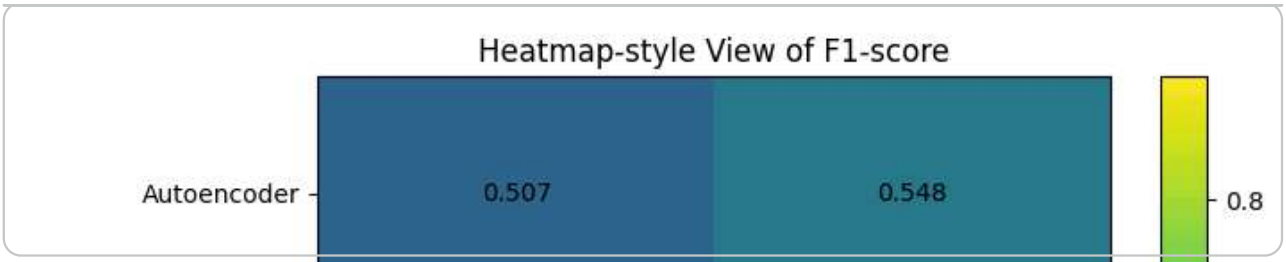
```
for dataset_name in advanced_comparison["Dataset"].unique():
    subset = advanced_comparison[advanced_comparison["Dataset"] == dataset_n

    plt.figure(figsize=(8, 6))
    plt.scatter(subset["Recall"], subset["Precision"], s=120)

    for _, row in subset.iterrows():
        plt.annotate(row["Model"], (row["Recall"], row["Precision"]), fontsi

    plt.title(f"Precision vs Recall Scatter Plot - {dataset_name}")
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.xlim(0, 1.02)
    plt.ylim(0, 1.02)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```



14. Chart 6 — Heatmap-style matrix for F1-score

```
heatmap_f1 = advanced_comparison.pivot(index="Model", columns="Dataset", val
fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(heatmap_f1.values, aspect="auto")

ax.set_xticks(np.arange(len(heatmap_f1.columns)))
ax.set_yticks(np.arange(len(heatmap_f1.index)))
ax.set_xticklabels(heatmap_f1.columns)
ax.set_yticklabels(heatmap_f1.index)
ax.set_title("Heatmap-style View of F1-score")

for i in range(heatmap_f1.shape[0]):
    for j in range(heatmap_f1.shape[1]):
        ax.text(j, i, f"{heatmap_f1.iloc[i, j]:.3f}", ha="center", va="cente

fig.colorbar(im)
plt.tight_layout()
plt.show()
```





15. Chart 7 — Model ranking plot

```
for dataset_name in rank_table["Dataset"].unique():
    subset = rank_table[rank_table["Dataset"] == dataset_name].sort_values("

    plt.figure(figsize=(10, 5))
    plt.plot(subset["Model"], subset["Rank"], marker="o")
    plt.gca().invert_yaxis()
    plt.title(f"Model Ranking by F1-score - {dataset_name}")
    plt.xlabel("Model")
    plt.ylabel("Rank (1 = Best)")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```